RAG — Part 2: Beyond the Basics

From "it works in a notebook" to "it survives in production"

Embedding Models

Pick once without re-indexing 3 times

Vector DBs

Scale without lock-in regret

SQL vs Vector DB

Different tools, different jobs

2025–2026 Advances

GraphRAG, HyDE, Agentic RAG, Hybrid Search

RAG in 2026

More relevant, not less — here is why

AGENTIC AI BUILDER EXPERT BOOTCAMP — BATCH 4.0

Recap: Where Part 1 Left Off

Part 1 built the minimal RAG loop: **Load → Chunk → Embed → Store → Retrieve → Ground → Generate.** That minimal loop is the skeleton. Everything in this deck is the muscle, tendons, and nervous system around it.

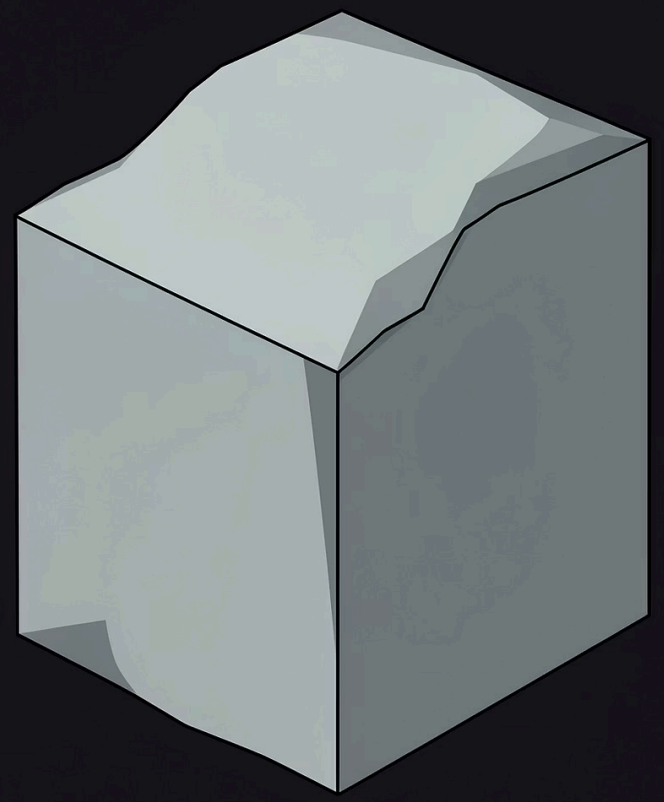
Part 1 Covered

- What RAG is and why it exists
- Embeddings and vector similarity
- Chunking strategies
- The minimal pipeline end-to-end
- Common failure modes

Part 2 Adds

- How to pick an embedding model without re-indexing 3 times
- How to pick a vector DB without lock-in regret
- Why your relational DB is the wrong tool for semantic search
- GraphRAG, HyDE, Agentic RAG, Rerankers, Hybrid Search
- Why RAG is more relevant in 2026, not less

ⓘ This deck assumes you have already built the minimal pipeline. We are not re-explaining embeddings or chunking — we are going deeper.



The Embedding Model Is Your Foundation

Your embedding model decides which chunks are "close" to a query. **Change it later and you re-embed your entire corpus. It is a one-way door.**

1

Quality

Measured on MTEB (Massive Text Embedding Benchmark), NDCG@10 for retrieval. The leaderboard is a starting point — not the final answer.

2

Dimensions

Higher = more storage, slower search, marginally better quality. 768–1,024 dims is the production sweet spot for most RAG systems.

3

Context Window

How long a chunk you can embed in one pass. Mismatching chunk size to model context window silently degrades quality.

⚠ Always benchmark on YOUR corpus with YOUR queries. MTEB scores on academic datasets do not transfer automatically to your domain.

Embedding Models: The 2026 Landscape

An honest map of the current space (April 2026). No single winner — the right choice depends on your constraints.

Commercial APIs (Managed, Closed)

Model	Dims	Context	Notes
Gemini Embedding 2	3,072	8K	Top MTEB, multimodal, Matryoshka
Voyage-3-large	1,024	32K	Strong retrieval, $0.12/1M tokens
OpenAI text-embedding-3-large	3,072	8,191	Ecosystem default, Matryoshka
Cohere embed-v4	1,024	128K	Enterprise multilingual, Rerank-native

Open Weights (Self-Host)

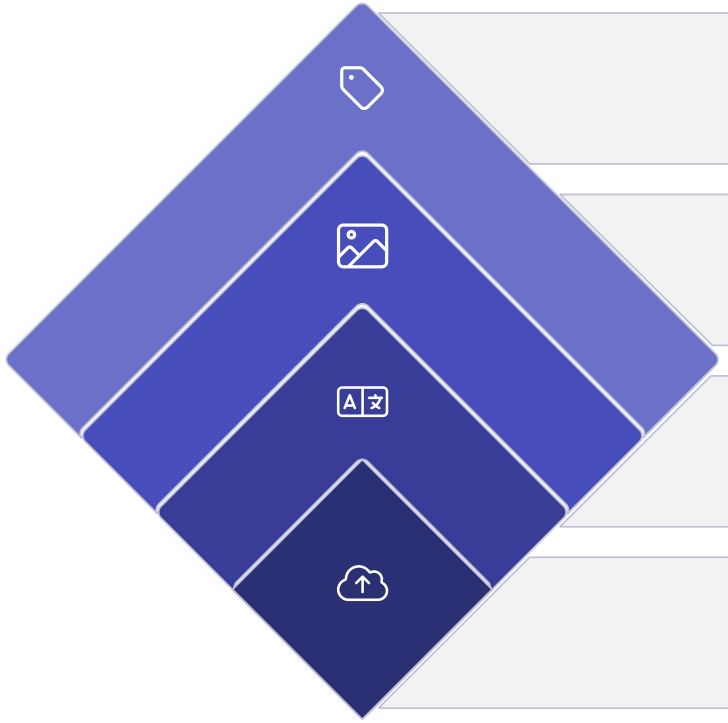
Model	License	Notes
Qwen3-Embedding-8B	Apache 2.0	Top multilingual MTEB
BGE-M3	MIT	Strong baseline, cheap to host
NV-Embed-v2	Non-commercial	Top open-weight English
Jina Embeddings v4	Apache 2.0	LoRA adapters, multimodal

Specialized Domains

- **Medical:** PubMedBERT, BioLORD
- **Finance:** Voyage Finance, BGE Financial Matryoshka
- **Legal:** Fine-tuned BGE / Qwen3 on legal corpora
- **Code:** Voyage code-3, Gemini Embedding 2 (MTEB Code 84.0)

Choosing an Embedding Model: Decision Tree

Stop agonizing. Ask these questions in order. The first constraint that applies wins.



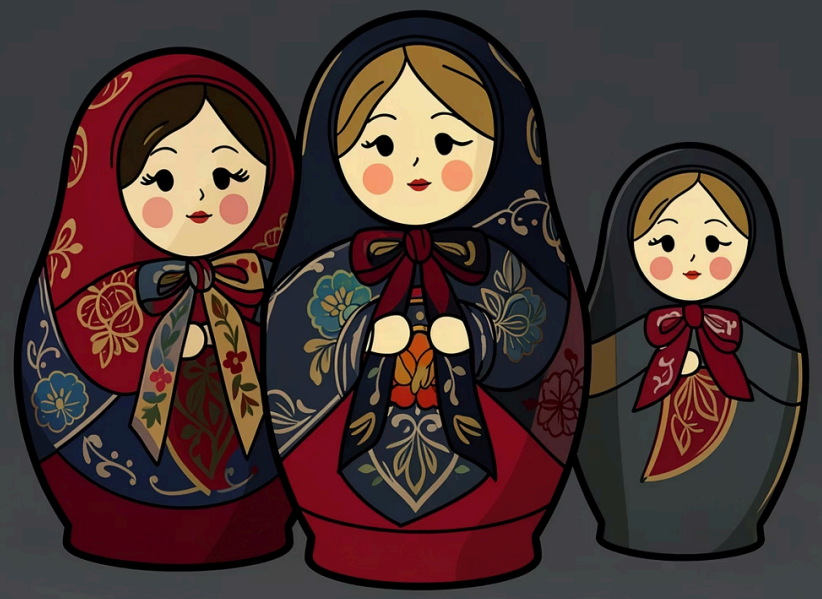
Chunk Size & Budget
Chunk length or budget first?

Multimodal Needed?
Text plus images or PDFs?

Language
English or multilingual?

Third-party Allowed?
Can you send data externally?

⚠ Non-negotiable: test on your corpus with recall@5, recall@10, and NDCG@10 before committing. A model that ranks #1 on MTEB may rank #5 on your domain.



Matryoshka: The Dimension Trick You Should Know

Modern embedding models — Gemini 2, Voyage 4, Cohere v4, OpenAI 3-*, Jina v5 — support **Matryoshka Representation Learning (MRL)**.

What It Means

Generate ONE embedding at full dimension (e.g., 3,072), then truncate to 768 or 256 at storage time with graceful quality decay — not catastrophic loss.

Why You Care

- **Storage:** 768 dims costs ~75% less than 3,072 dims
- **Speed:** Similarity search runs faster on shorter vectors
- **Flexibility:** Decide storage tier per use case without re-embedding

3,072

Full Dims

Maximum quality, maximum cost

1,024

Sweet Spot

Production default for most RAG systems

75%

Storage Saved

768 vs 3,072 dims at same model

What Is a Vector Database, Really?

A vector database is an index optimized for one job: **finding the nearest vectors to a query vector, fast, at scale.**

It Is NOT

- A replacement for your system of record
- A general-purpose database
- Just "a table with a vector column"

Core Capabilities That Define a Real Vector DB

- **ANN indexes** — HNSW, IVF, DiskANN for fast approximate search
- **Metadata filtering** alongside vector search
- **Hybrid search** — sparse (keyword) + dense (vector) fusion
- **Scale** — billions of vectors with sub-100ms retrieval
- **Multi-tenancy and ACLs** for enterprise deployments

Vector Databases: The 2026 Landscape

No single winner. Pick based on your constraints — scale, filtering needs, deployment model, and cost structure.

Managed Cloud (Easiest to Start)

DB	Strengths
Pinecone	Best DX, fully managed, integrated inference API
Weaviate Cloud	Hybrid search native, multi-tenancy, BYO vectorizer
Qdrant Cloud	Rust-based, strong filtering, generous free tier

Hybrid Databases (Not Purpose-Built)

DB	Use When
pgvector (Postgres)	Already on Postgres, modest scale
Elasticsearch / OpenSearch	BM25 + dense, strong hybrid
MongoDB Atlas Vector	Already on Mongo

Open-Source, Self-Hostable

DB	Strengths
Milvus	43K+ GitHub stars, billion-scale, MRL support
Weaviate	Module ecosystem, BYO embedder + reranker
Qdrant	Single-binary Rust, excellent filtering, lightweight
Chroma	Dev-friendly, embeddings-first, great for prototypes

Rule of thumb: prototype on Chroma, graduate to Milvus, Pinecone, or Qdrant at scale.

Choosing a Vector DB: The Decision Matrix

What matters beyond "which one is fastest." Match your constraints to the right tool.

Constraint	Threshold	Recommended
Scale	Under 1M vectors	Anything — Chroma, pgvector, Qdrant
Scale	1M–100M vectors	Qdrant, Weaviate, Milvus
Scale	Over 100M vectors	Milvus, Pinecone, Vespa
Filtering	Heavy metadata filters (tenant_id, date, category)	Qdrant or Milvus
Hybrid Search	BM25 + vector fusion required	Weaviate, Elasticsearch, Milvus 2.4+
Deployment	Air-gapped / on-prem	Milvus, Qdrant, Weaviate self-hosted
Cost Model	Storage-heavy workload	DiskANN-backed: Milvus, Vespa
Cost Model	Query-heavy workload	In-memory HNSW

LangChain/LlamaIndex support is near-universal now. Pick for ops maturity, not framework integration.

SQL DB vs Vector DB: Different Tools, Different Jobs

This confuses a lot of beginners. They are not competitors — they are complements. Each is optimized for a fundamentally different type of question.

Relational / SQL DB

Postgres, MySQL, SQL Server

- Stores structured rows and columns
- Optimized for exact matches, joins, aggregations, transactions (ACID)
- Query with WHERE clauses, B-tree / hash indexes

Answers: **"Find all orders from customer X in Q3 2025 over $500"**

Vector DB

Milvus, Pinecone, Qdrant, Weaviate

- Stores vectors + text + metadata
- Optimized for semantic similarity (ANN) at scale
- Query with a vector, returns top-k nearest neighbors

Answers: **"Find chunks semantically similar to 'how does bread rise'"**

SQL answers

"What EXACTLY matches?"

Vector DB answers

"What MEANS the same thing?"

Production reality

80% of serious RAG systems run both side-by-side

Side-by-Side: How the Same Data Is Stored

Take one customer support ticket as a concrete example. The same underlying information is stored and indexed in fundamentally different ways.

In SQL (Postgres)

```
ticket_id   = 10023
customer_id = 412
created_at  = 2026-03-14
status      = "open"
subject     = "Login error on mobile"
body        = "I cannot sign in on iOS..."

Indexed on:
  ticket_id   (primary key)
  customer_id (foreign key)
  created_at  (B-tree)
  status      (hash)
```

Best for: **"Show me all open tickets for customer 412 this week."**

In Vector DB (Milvus / Qdrant)

```
id        = "10023_chunk_0"
text      = "I cannot sign in on iOS..."
embedding = [0.012, -0.441, ..., 0.087]
            (1,024 dims)
metadata  = {
  ticket_id:   10023,
  customer_id: 412,
  status:      "open",
  created_at:  "2026-03-14"
}

Indexed on:
  embedding (HNSW)
  metadata filters
```

Best for: **"Find other tickets semantically similar to this login issue, across any customer."**

ⓘ Real production RAG systems use both. SQL for identity, joins, ACLs, audit. Vector DB for semantic retrieval.

When to Use What: A Decision Guide

Use SQL / Relational When

- You need exact matches, joins, aggregations
- Transactions matter (ACID)
- Data is highly structured
- You need audit trails and referential integrity

Use Vector DB When

- Query is natural language, answer may phrase it differently
- You want "similar" not "identical"
- Corpus is unstructured text, images, audio, PDFs
- You are building RAG, semantic search, recommendations, dedup

Use BOTH When

- Fine-grained permissions per document (SQL holds ACLs, vector DB holds embeddings)
- You need to join retrieved chunks back to authoritative records
- Hybrid: "find tickets similar to X AND opened in last 7 days AND from Enterprise tier"

Production reality: 80% of serious RAG systems run SQL + Vector DB side-by-side. This is not over-engineering — it is the correct architecture.

Hybrid Search: Keyword + Vector Together

In 2026, hybrid search is table stakes. If your RAG is only dense-vector, you are leaving retrieval quality on the table.

Pure Vector Search Fails On

- Product codes, SKUs, invoice numbers (exact tokens matter)
- Named entities with no semantic context
- Rare technical jargon the embedding model never saw

Pure Keyword (BM25) Fails On

- Paraphrased questions ("bread rising" vs "dough expanding")
- Cross-lingual queries
- Conceptual questions with no exact token overlap

Hybrid Search = Run Both, Fuse Rankings

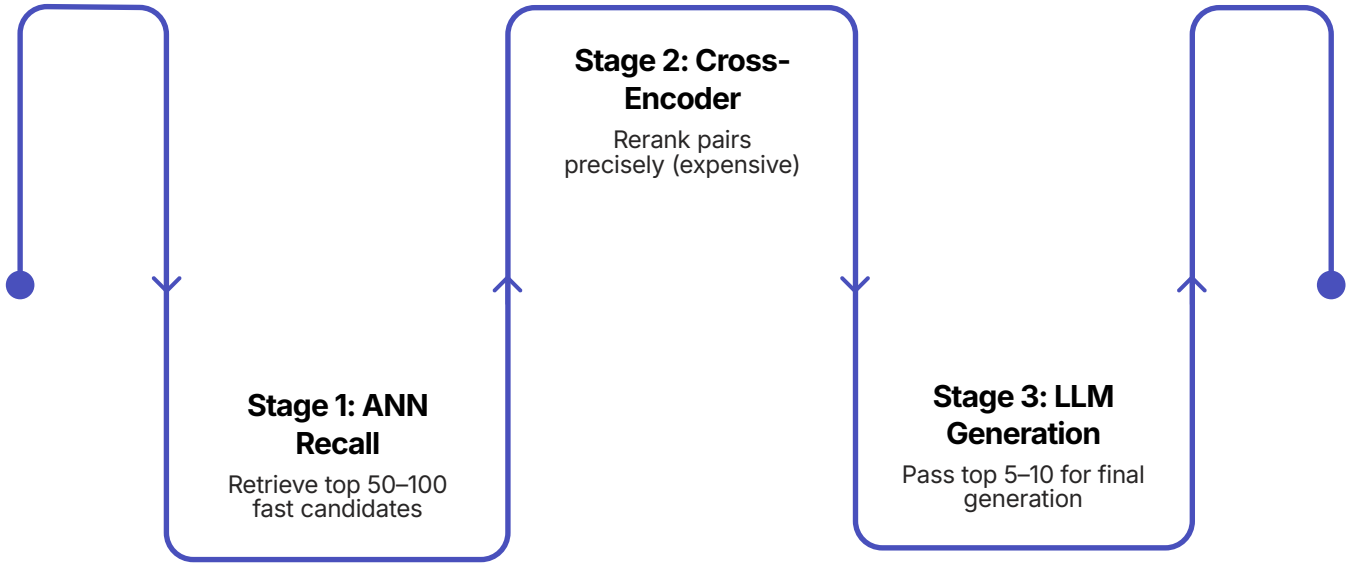
The standard fusion algorithm is **Reciprocal Rank Fusion (RRF)**: combine the ranked lists from BM25 and dense vector search into a single merged ranking.

Native hybrid support in 2026:

- Weaviate
- Milvus 2.4+
- Elasticsearch / OpenSearch
- Qdrant
- Pinecone

Rerankers: The Second Pass That Saves You

Retrieval is cheap and approximate. Rerankers are expensive and precise. The two-stage approach gives you the best of both.



Production Rerankers in 2026

- Cohere Rerank 3
- Voyage rerank-2
- BGE-Reranker-v2 (open weights)
- Jina Reranker v2

Impact

10–30

Points Lift

Percentage point improvement in recall@5 is common

Often the **cheapest quality improvement** in a RAG stack. Add a reranker before you add a better LLM.

GraphRAG: When Relationships Matter

Traditional RAG treats your corpus as a flat bag of chunks. It cannot answer "what are the main themes across these 500 documents?" or "who reports to whom, transitively?"

What GraphRAG Builds

- **Entities** (people, companies, products, concepts) = nodes
- **Relationships** (reports_to, acquired, mentioned_with) = edges
- **Retrieval** traverses the graph alongside vector search

Why It Matters

- Global reasoning over a corpus, not just local lookup
- Multi-hop questions ("products launched by companies acquired by X since 2023")
- Traceable, auditable answers — you can show the reasoning path

Practical Guidance

Graph construction is expensive. Microsoft's GraphRAG is the reference implementation.

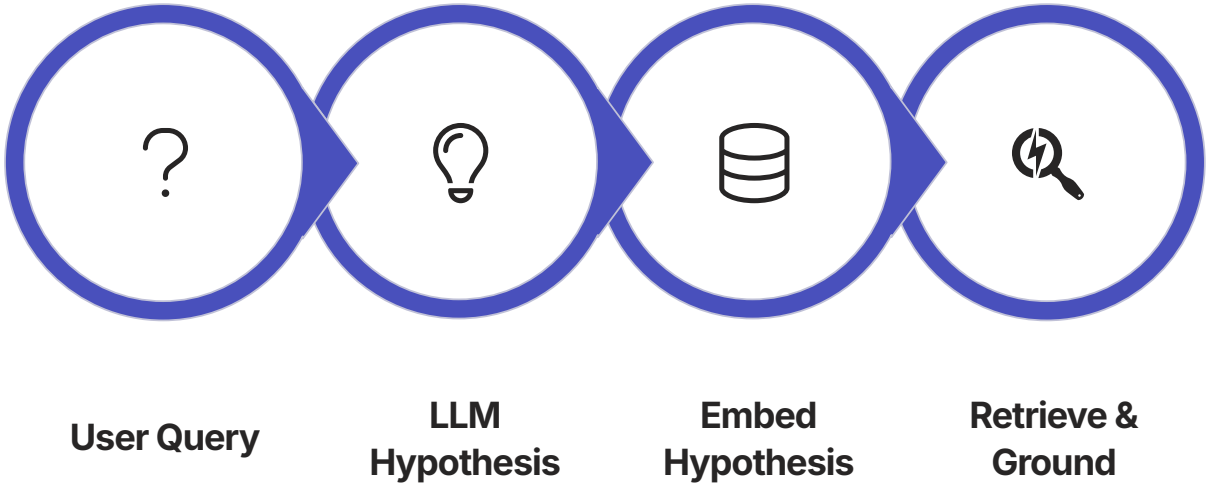
Pair with vector RAG. Do not replace it.

GraphRAG excels at:

- Thematic summarization across large corpora
- Multi-hop entity traversal
- Regulated industries needing traceable reasoning paths

HyDE: Let the LLM Help the Retriever

Hypothetical Document Embeddings (HyDE) — for vague or sparse queries where the question itself is too generic to match anything useful.



The Problem It Solves

User asks "Why is my build slow?" — too generic to match anything useful in a dense vector index. The query embedding lands in a vague region of the vector space.

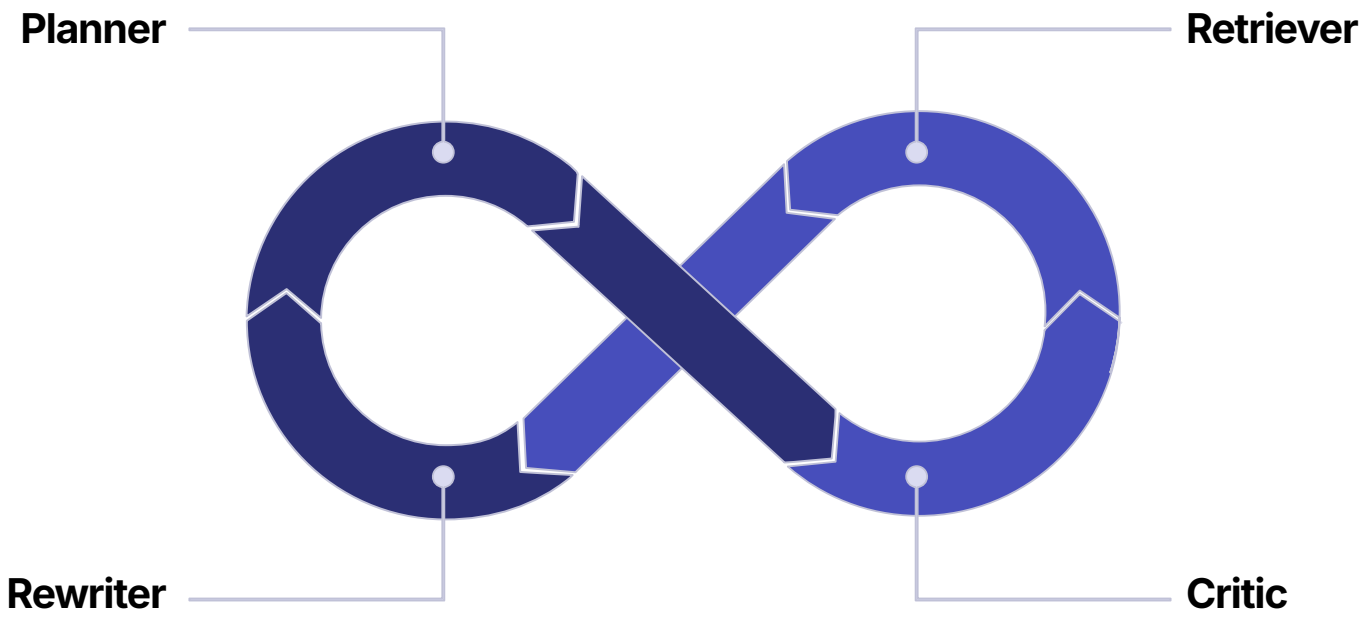
The Trade-off

Works surprisingly well on underspecified queries. Adds **one LLM call** to the query path. The hallucination is never shown to the user — it is purely a retrieval aid.

HyDE is not a default — it is a targeted fix for sparse or underspecified queries. Measure before adding it to every query path.

Agentic RAG: The 2026 Default

Static RAG is one pass with no feedback. Agentic RAG is a loop of agents that plan, retrieve, critique, rewrite, and re-retrieve until confident — or until they hit a budget.



This is a cyclic, stateful graph — not a linear pipeline. Built in LangGraph. Human-in-the-loop checkpoints are first-class citizens, not afterthoughts.

Agentic RAG vs GraphRAG vs Long Context

The three forces reshaping RAG in 2026. They are not substitutes — they solve different problems.

Long Context Windows

Claude and Gemini now handle 1M+ tokens. For corpora under ~750K words, you may not need RAG at all — just stuff the docs into context and ask.

Works. Expensive at scale. Attention quality degrades across very long inputs.

GraphRAG

Structured entity/relationship retrieval. Best for thematic, multi-hop, global reasoning over large corpora.

Expensive to build. Irreplaceable for traceable multi-hop answers.

Agentic RAG

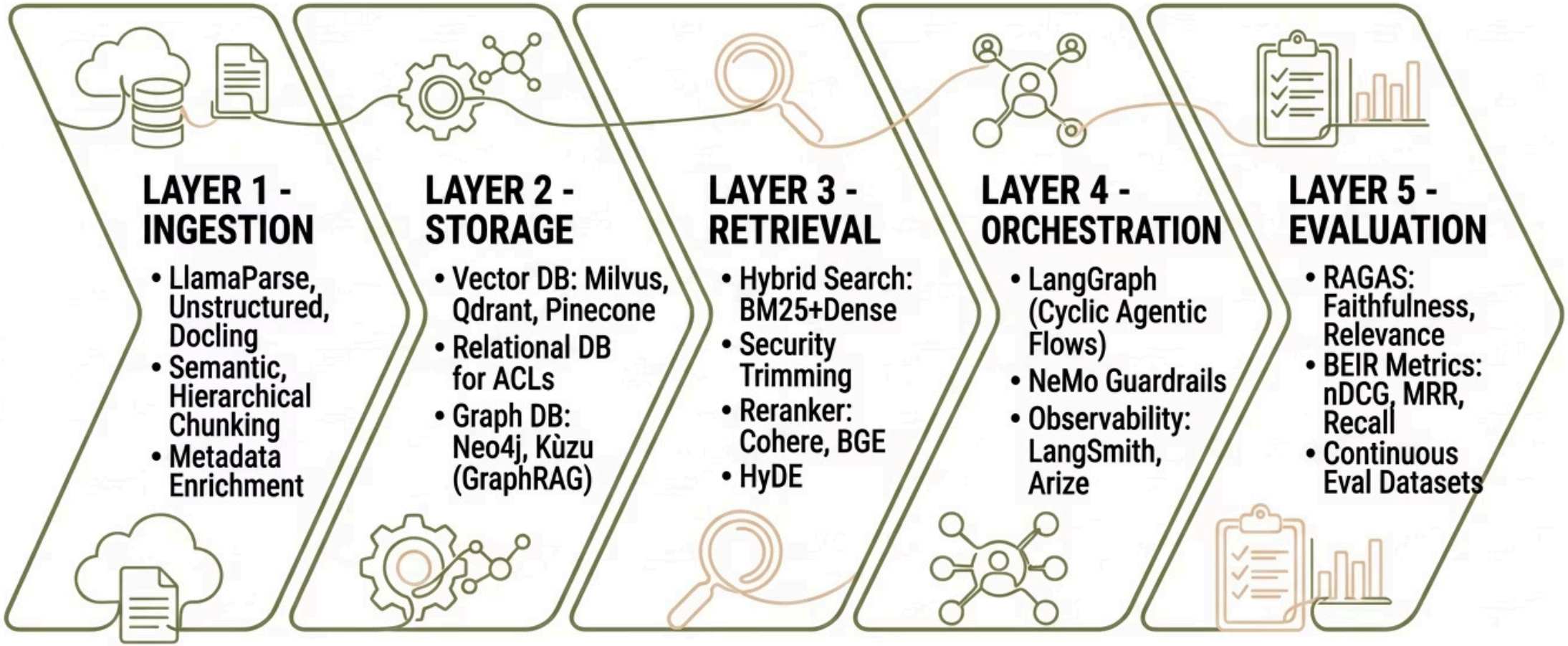
Autonomous agents that loop over retrieval. Best for complex, open-ended, multi-step research tasks.

Agentic RAG does not kill GraphRAG. They solve different problems.

> "You cannot compute your way out of a topological deficit." — NYU Shanghai study, April 2026. If your data lacks explicit structure, no amount of agent cleverness can perfectly reconstruct it on the fly.

The 2026 Production RAG Stack

A production-grade RAG stack today is not one vendor, one flow. It is a layered architecture where each layer has a specific job.



Evaluation: The Part Most Teams Skip

Retrieval quality determines answer quality. You cannot improve what you do not measure. Build an eval dataset before you ship.

Retrieval Metrics

Metric	What It Measures
Recall@K	Fraction of relevant chunks in top K results
NDCG@10	Rank-aware relevance — rewards relevant chunks ranked higher
MRR	Mean reciprocal rank of first relevant chunk

Generation Metrics (RAGAS)

Metric	What It Measures
Faithfulness	Does the answer stick to retrieved context?
Answer Relevance	Does the answer actually address the question?
Context Precision	Are retrieved chunks actually relevant?
Context Recall	Did retrieval find all the needed chunks?

⚠ 50–100 (question, ground-truth-answer, expected-source-docs) triples is enough to catch regressions. Regression testing beats vibes. Every single time.

Why RAG Is More Relevant in 2026, Not Less

Every few months someone declares "RAG is dead." Every time, they are wrong.

1 Fresh Knowledge

LLMs still have a training cutoff. Your docs change daily. Retraining is not on the table.

2 Private Data

Your contracts, Slack, CRM, Jira, HRIS will never be in a foundation model. They should not be.

3 Access Control

You need per-user, per-document permissions. RAG enforces this at query time. Fine-tuning cannot.

4 Traceability and Auditability

Regulated industries (finance, health, legal, public sector) need source citations. GraphRAG + RAG deliver.

5 Long Context Is Not a Silver Bullet

1M-token context is expensive per query, slow, and attention quality degrades across very long inputs. RAG is still cheaper for most scaled workloads.

6 Agentic AI Needs Grounded Retrieval

Every serious AI agent needs a knowledge runtime. That runtime is RAG — or its agentic descendant.

Where RAG Is Heading in 2026 and Beyond

The line between "RAG system" and "AI agent" is dissolving. That is the point.

Multimodal RAG

Retrieval across text, images, video, audio, PDFs in one vector space. Gemini Embedding 2, Qwen3-VL, Voyage Multimodal are leading this.

Late Interaction Retrieval

ColBERT-style token-level matching for precision. Scores every token pair between query and document — more expensive, more accurate.

RAG-as-Orchestration

RAG as the knowledge runtime layer, with retrieval quality gates, source verification, and governance embedded in every operation.

RL-Trained Retrieval

Search-R1, Graph-R1 style agents trained end-to-end to retrieve. The retriever itself learns from reward signals, not just supervised labels.

Edge RAG

On-device vector DBs + small embeddings for privacy-first deployments. No data leaves the device.

Self-Updating Indexes

CDC (change data capture) pipelines that keep vector indexes fresh in near real-time. No more stale retrieval from batch re-indexing.

Common Anti-Patterns to Avoid

Things I see every cohort of Batch 4.0 do wrong. These are not edge cases — they are the default mistakes.

Picking an embedding model from a blog post

Not from eval on your corpus. MTEB rankings on academic datasets do not transfer to your domain automatically.

Using pgvector at 100M+ vectors

Because "we already have Postgres." pgvector is a fine starting point. It is not a billion-scale vector DB.

Chunking by fixed token count

Ignoring document structure. A 512-token chunk that splits a table in half is worse than a 900-token chunk that keeps it intact.

Skipping metadata

No source, no date, no author, no permissions. You cannot filter what you did not store. You cannot audit what you did not track.

No reranker

Leaving 10–30 percentage points of recall on the floor. A reranker is often the cheapest quality improvement in the entire stack.

No eval dataset — shipping, then hoping

Treating RAG as a project, not a product. No monitoring, no regressions, no re-indexing cadence.

Assuming the LLM will "figure it out"

When retrieval is bad, the LLM will not save you. It will hallucinate confidently. Bad retrieval in, bad answers out. Every single time.

What You Should Walk Away With

RAG in 2026 is no longer a single technique. It is a stack. The teams that will ship reliable AI systems are the ones that treat RAG as engineering — not magic.

01

Embedding Model

Chosen for your corpus, not a leaderboard. Benchmarked on recall@5, recall@10, NDCG@10 against your actual queries.

02

Vector DB

Sized for your scale, filtering requirements, and deployment constraints. Not chosen because it was in a tutorial.

03

SQL DB Alongside It

For identity, ACLs, joins, and audit. The vector DB does not replace your system of record.

04

Hybrid Search as the Default

Keyword + dense, fused with RRF. Not the exception — the baseline.

05

Reranker Before the LLM

Two-stage retrieval: cheap ANN first, precise cross-encoder second. Top 5–10 to the LLM.

06

GraphRAG + Agentic RAG Where Needed

GraphRAG where relationships matter. Agentic RAG where one pass is not enough.

07

Eval Metrics, Not Vibes

RAGAS + BEIR metrics. An eval dataset before you ship. Regression testing as a continuous practice.

Resources and Where to Go Next

Benchmarks

- **MTEB Leaderboard** — huggingface.co/spaces/mteb/leaderboard
- **BEIR** — retrieval benchmark suite
- **RAGAS** — github.com/explodinggradients/ragas

Reading

- Microsoft GraphRAG paper and repo
- LangGraph documentation
- Anthropic's "Contextual Retrieval" blog post
- LlamaIndex RAG techniques guide

Frameworks to Try

LangGraph

Agentic orchestration

LlamaIndex

Ingestion + retrieval abstractions

Haystack

Production pipelines

Milvus / Qdrant / Weaviate

Vector DBs

This deck continues the Agentic AI Builder Expert Bootcamp, Batch 4.0 — where we build production-grade RAG and agentic systems end-to-end. If you want to stop reading about it and start shipping it, you know where to find me.

> **Learn to build agents before agents learn to replace you.**